Handwritten:

```
add_ref(unsigned long, RefVec*, RefVec const*, RefVec const*):
    test   rdi, rdi
    je     .L8
    sal    rdi, 4
    xor    eax, eax
.L3:
    vmovups xmm0, XMMWORD PTR [rdx+rax]
    vaddps  xmm0, xmm0, XMMWORD PTR [rcx+rax]
    vmovups XMMWORD PTR [rsi+rax], xmm0
    add    rax, 16
    cmp    rax, rdi
    jne    .L3
.L8:
    ret
```

Created:

```
add_vmlib(unsigned long, SmallVec<4ul, float>*, SmallVec<4ul, float> const*, SmallVec<4ul, float>
const*):
    test   rdi, rdi
    je     .L8
    sal    rdi, 4
    xor    eax, eax
.L3:
    vmovups xmm0, XMMWORD PTR [rcx+rax]
    vaddps  xmm0, xmm0, XMMWORD PTR [rdx+rax]
    vmovups XMMWORD PTR [rsi+rax], xmm0
    add    rax, 16
    cmp    rdi, rax
    jne    .L3
.L8:
    ret
```

the line that runs operator+ in release:

```
    vaddps  xmm0, xmm0, XMMWORD PTR [rdx+rax]
```

the line that does the operator+ in debug:

```
        call   SmallVec<4ul, float> operator+<4ul, float>(SmallVec<4ul, float> const&, SmallVec<4ul,
float> const&)
```

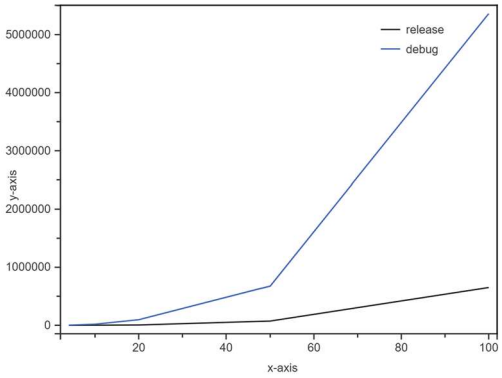
Questions:

The main differences are the fact that the main code is stripped back in the release version. It uses vaddps which is a pre set 'function' which will basically do a for loop straight into memory whereas the debug code isn't made together into the assembly code and is more broken down into different sections. This basically means that the release is a lot more optimised of everything within the code. Later in the code the debug runs vaddss which is a single memory version of vaddps.

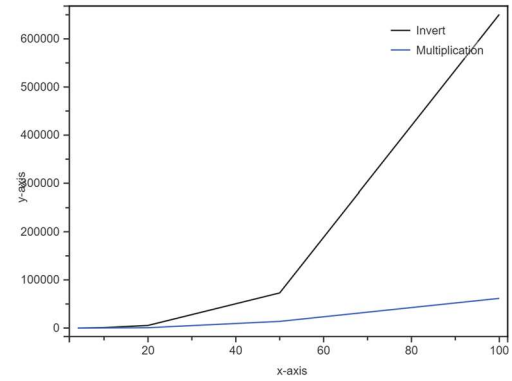
The difference between ref and vmlib are almost the complete same apart from swapping some of the locations for example rdi and rax.

Overall, when you use the -02 optimiser it will strip everything down to the same structure so there is no major difference between the hand-written and what I coded.

A)



B)



The reason that they are similar for when the size is 4 is due to the fact that when tN = 4 it is overridden by a hard coded version so that it will skip the code that I made. This means it doesn't have to go into a maze of multiple for loops which go through jump between rows and columns of the data. However, the release multiplication is the most efficient compared to any of the invert versions especially once tN > 4 as nothing is hard coded

My computer specs:

Processor	Intel(R) Core(TM) Ultra 5 125H (3.60 GHz)
Installed RAM	16.0 GB (15.4 GB usable)
Graphics card	Intel(R) Arc(TM) Graphics (128 MB)
System type	64-bit operating system, x64-based processor